

LU Factorization

Geometric Algorithms
Lecture 12

Matrix Factorization

Matrix Factorization

A **factorization** is an equation which expresses a matrix as a product of two or more matrices.
Factoring matrix can:

Matrix Factorization

A **factorization** is an equation which expresses a matrix as a product of two or more matrices.

Factoring matrix can:

- » make computing with A faster

Matrix Factorization

A **factorization** is an equation which expresses a matrix as a product of two or more matrices.

Factoring matrix can:

- » make computing with A faster
- » make working with A easier

Matrix Factorization

A **factorization** is an equation which expresses a matrix as a product of two or more matrices.

Factoring matrix can:

- » make computing with A faster
- » make working with A easier
- » expose important information about A

Matrix Factorization

A **factorization** is an equation which expresses a matrix as a product of two or more matrices.

Factoring matrix can:

» make computing with A faster **LU Decomposition**

» make working with A easier

» expose important information about A

The Problem

Solve the equations

$$A\mathbf{x} = \mathbf{b}_1 \quad , \quad A\mathbf{x} = \mathbf{b}_2 \quad \dots \quad A\mathbf{x} = \mathbf{b}_{k-1} \quad , \quad A\mathbf{x} = \mathbf{b}_k$$

We want to solve several matrix equations over the same matrix

The Problem

Solve for X in the equation $AX = B$ where X and B are matrices of appropriate dimension

This is (essentially) the same question

The Problem

Solve for X in the equation $AX = B$ where X and B are matrices of appropriate dimension

If A is invertible, then $X = A^{-1}B$

What if A^{-1} is not invertible? Even if it is, can we do it faster?

LU Factorization at a High Level

Given a $m \times n$ matrix A , we are going to factorize A as

echelon form of A

$$A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix}}_L \underbrace{\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_U$$

LU Factorization at a High Level

Given a $m \times n$ matrix A , we are going to factorize A as

echelon form of A

$$A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix}}_L \underbrace{\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_U$$

Note. This applies to non-square matrices

What are "L" and "U"?

L stands for "lower" as in *lower triangular*

U stands for "upper" as in *upper triangular*

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix} \begin{bmatrix} \blacksquare & * & * & * \\ 0 & \blacksquare & * & * \\ 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & \blacksquare \end{bmatrix}$$

L U

The Fundamental Question

$$A = LU$$

echelon form of A

The Fundamental Question

$$A = LU$$

echelon form of A

We know how to build U , that's just the forward phase of Gaussian elimination

The Fundamental Question

$$A = LU$$

echelon form of A

We know how to build U , that's just the forward phase of Gaussian elimination

How do we build L ?

The Fundamental Question

$$A = LU$$

echelon form of A

We know how to build U , that's just the forward phase of Gaussian elimination

How do we build L ?

The idea. L "implements" the row operations of the forward phase

Elementary Matrices

Recall: Elementary Row Operations

scaling	multiply a row by a number
interchange	switch two rows
replacement	add a scaled equation to another

Elementary row operations are linear, so they are implemented by matrices

General Elementary Scaling Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

General Elementary Scaling Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

If we want to perform $R_3 \leftarrow kR_3$ then we need the identity matrix but with the entry $A_{33} = k$.

General Elementary Scaling Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

If we want to perform $R_3 \leftarrow kR_3$ then we need the identity matrix but with the entry $A_{33} = k$.

If we want to perform $R_i \leftarrow kR_i$ then we need the identity matrix but with the entry $A_{ii} = k$.

General Replacement Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ k & 0 & 0 & 1 \end{bmatrix}$$

General Replacement Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ k & 0 & 0 & 1 \end{bmatrix}$$

If we want to perform $R_4 \leftarrow R_4 + kR_1$, then we need the identity matrix but with the entry $A_{41} = k$.

General Replacement Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ k & 0 & 0 & 1 \end{bmatrix}$$

If we want to perform $R_4 \leftarrow R_4 + kR_1$, then we need the identity matrix but with the entry $A_{41} = k$.

If we want to perform $R_i \leftarrow R_i + kR_j$, then we need the identity matrix but with the entry $A_{ij} = k$.

General Swap Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

If we want to swap R_2 and R_3 , then we need the identity matrix, but with R_2 and R_3 swapped.

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \dots \sim E_k E_{k-1} \dots E_2 E_1 A$$

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \dots \sim E_k E_{k-1} \dots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

$$B = (E_k E_{k-1} \dots E_2 E_1) A = EA$$

where E implements a sequence of row operations. So:

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \dots \sim E_k E_{k-1} \dots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

$$B = \overset{\text{Invertible}}{(E_k E_{k-1} \dots E_2 E_1)} A = EA$$

where E implements a sequence of row operations. So:

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \dots \sim E_k E_{k-1} \dots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

$$B = \overset{\text{Invertible}}{(E_k E_{k-1} \dots E_2 E_1)} A = EA$$

where E implements a sequence of row operations. So:

$$A = E^{-1} B = (E_1^{-1} E_2^{-1} \dots E_{k-1}^{-1} E_k^{-1}) B$$

LU Factorization Algorithm

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization( $A$ ):  
2       $L \leftarrow$  identity matrix  
3       $U \leftarrow A$ 
```


LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps  
5      FOR each row operation OP in the prev step:
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps  
5      FOR each row operation OP in the prev step:  
6          E ← the matrix implementing OP
```


LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps  
5      FOR each row operation OP in the prev step:  
6          E ← the matrix implementing OP  
7          L ← L @ E-1      # note the multiplication on the right
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps  
5      FOR each row operation OP in the prev step:  
6          E ← the matrix implementing OP  
7          L ← L @ E-1      # note the multiplication on the right  
8      RETURN (L, U)
```


LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):  
2      L ← identity matrix  
3      U ← A  
4      convert U to an echelon form by GE forward step # without swaps  
5      FOR each row operation OP in the prev step:  
6          E ← the matrix implementing OP  
7          L ← L @ E-1      # note the multiplication on the right  
8      RETURN (L, U)      we'll see how to do this more efficiently
```

The forward part of Gaussian
elimination is matrix
factorization

The "L" Part

$$E = E_k E_{k-1} \dots E_2 E_1$$

This a product of elementary matrices

So $L = E^{-1} = E_1^{-1} E_2^{-1} \dots E_{k-1}^{-1} E_k^{-1}$ **!! the order reverses !!**

We won't prove this, but it's worth thinking about: **why is this lower triangular?**

And can we build this in a more efficient way?

workshop